

Batching and Paged Attention

Serving many users from one model · video 2:11

This lesson answers one question: how does one model serve many users efficiently? **Batching** lets one read of the weights serve many requests: GPT-2 went from 26 to 404 tokens per second.

Continuous batching refills the batch as requests finish, instead of wasting 65% of the work on padding. **Paged attention** stores the KV cache in small pages allocated as needed, so the same memory holds 2.5 times as many requests.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 5).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Batching	▶ 0:08-0:51	2	☐
2. Uneven requests and continuous batching	▶ 0:51-1:16	1	☐
3. Paged attention	▶ 1:16-2:02	3	☐

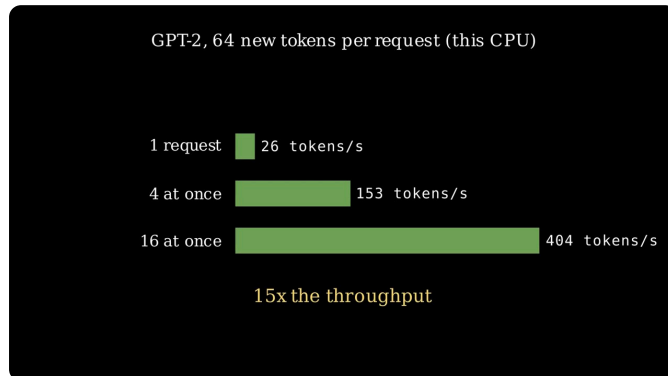
Before you start: Deep Dive episodes 21 (batch size) and 31 (the KV cache). Code: `code/d32_batching_paging` (GPT-2 small, about 500 MB).

CONCEPT 1 OF 3

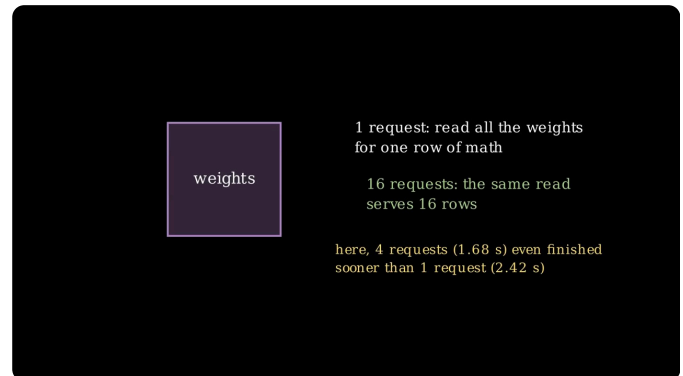
Batching

▶ Video 0:08-0:51

A model answering one user at a time wastes most of its hardware. GPT-2 writing 64 tokens for one request manages **26 tokens per second** on this CPU; four requests at once, **153**; sixteen, **404**: about 15 times the throughput.



▶ 0:35 64 tokens per request: 26 tokens/s alone, 153 with 4 requests, 404 with 16.



▶ 0:50 Decoding reads all the weights for little math; a batch reuses that read for many requests.

Why? Decoding one token reads all the weights for very little math (episode 31). With a batch, the same read serves many requests in one pass. On this CPU a batch of one is so inefficient that four requests (1.68 s) even finished sooner than one (2.42 s).

KEY IDEA

Batching turns memory-bound decoding into useful parallel work.

Check 1: Batching

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

With 16 requests at 404 tokens per second in total, how many tokens per second does each user see?

1.2 TRUE OR FALSE

“Batching makes each individual request much faster.”

True ☐ False ☐

CONCEPT 2 OF 3

Uneven requests and continuous batching

▶ Video 0:51-1:16

Real requests have different lengths. Sixteen requests wanting 20 to 1,000 tokens in a fixed batch: the batch runs until the longest finishes, $16 \times 1,000 = 16,000$ token slots for 5,570 useful tokens, **65% wasted** on requests that are already done.



▶ 1:04 16 requests of 20 to 1,000 tokens in a fixed batch: 16,000 slots, 5,570 useful, 65% wasted.



▶ 1:15 Continuous batching: finished requests leave, waiting ones join at the next step.

Continuous batching fixes that: as soon as a request finishes it leaves, and a waiting request joins at the next step. The batch stays full of useful work.

KEY IDEA

Schedule per step, not per batch.

Check 2: Uneven requests and continuous batching

Answer all 1 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

Two requests of 100 and 900 tokens in one fixed batch: what fraction of the slots is wasted?

.....

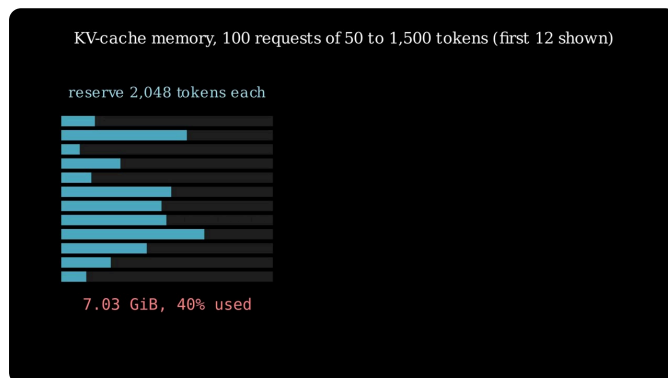
CONCEPT 3 OF 3

Paged attention

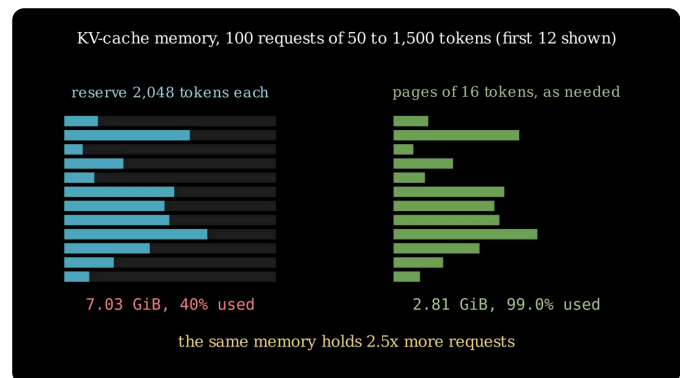
▶ Video 1:16–2:02

The other limit is memory: every request needs its KV cache. Reserving room for the maximum length (2,048 tokens) for each of 100 requests of 50 to 1,500 tokens (average 810), in GPT-2's float16 cache, takes **7.03 GiB**, and only **40%** is ever used.

Paged attention borrows an idea from operating systems: store the cache in small pages of 16 tokens, allocated only as a request grows, and keep a table of where each request's pages live. The same 100 requests need **2.81 GiB**, 99.0% used: the same memory holds 2.5 times as many requests.



▶ 1:33 Reserving 2,048 tokens for every request: 7.03 GiB, 40% used.



▶ 1:52 Pages of 16 tokens as needed: 2.81 GiB, 99.0% used: 2.5x more requests in the same memory.

KEY IDEA

Allocate the KV cache in small pages on demand; waste is at most one partly filled page per request.

Check 3: Paged attention

Answer all 3 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

How many 16-token pages does a 1,000-token request need, and how many token slots are wasted?

3.2 CALCULATION

GPT-2's float16 cache is 36,864 bytes per token. How much does reserving 2,048 tokens cost per request?

3.3 HANDS-ON CODE

Try it yourself. Compute the pages needed with the episode's rounding.

```
PAGE = 16
for n in (1, 16, 17, 1000):
    print(n, -(-n // PAGE))
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Batching · rewatch ▶ 0:08-0:51

1.1 About 25. $404 / 16 \approx 25$: nearly the single-request speed, for 16 users at once.

1.2 False. It raises total throughput; each request runs at about the same speed (here ~ 25 vs 26 tokens per second).

2. Uneven requests and continuous batching · rewatch ▶ 0:51-1:16

2.1 About 44%. $2 \times 900 = 1,800$ slots for 1,000 useful tokens.

3. Paged attention · rewatch ▶ 1:16-2:02

3.1 63 pages; 8 slots. $\lceil 1,000 / 16 \rceil = 63$ pages = 1,008 slots.

3.2 72 MiB. $36,864 \times 2,048 = 75,497,472$ bytes.

3.3 1 → 1, 16 → 1, 17 → 2, 1000 → 63. $-(n // \text{PAGE})$ is integer division rounded up.